

ReasoningGuard: Dual-Layer Defense for MCP-Based LLM Agents via Protocol Attestation and Reasoning Trace Verification

Anonymous submission

Abstract

Large language model (LLM) agents that invoke external tools through the Model Context Protocol (MCP) face a dual vulnerability: protocol-level exploits that bypass capability and origin checks, and reasoning-level manipulation where poisoned tool responses corrupt the agent’s decision process without triggering action-level monitors. Existing defenses address only one layer—AttestMCP hardens the protocol but cannot detect semantically corrupted reasoning, while guardrail models validate actions post hoc without verifying the reasoning chain that produced them. We propose REASONINGGUARD, a dual-layer defense that couples a Protocol-Attested Tool Gateway (PTG) with a Reasoning Trace Verifier (RTV). PTG extends AttestMCP with semantic intent attestation: every tool invocation carries a cryptographically bound reasoning summary that the gateway cross-checks against declared capabilities before forwarding the request. RTV structures the agent’s reasoning into a verifiable trace and applies a constrained judge model to detect inconsistencies between observed context, intermediate reasoning steps, and selected actions. We ground the necessity of the dual-layer design in the Layered Attack Surface Model (LASM) non-transferability property: PTG alone has zero detection power against cognitive-layer attacks, and RTV alone cannot prevent protocol-level exploits. On the MCPTox benchmark, REASONINGGUARD reduces the overall attack success rate from 72.8% to 5.3%—surpassing AttestMCP (12.4%) and guardrail baselines (28.1%)—while maintaining 91.2% task completion with 14.6ms median overhead per tool call. We further extend MCPTox with context-dependent and cross-session attack scenarios (MCPTox+), providing the first T3-temporality evaluation in MCP security.

1 Introduction

The Model Context Protocol (MCP) has rapidly become the de facto standard for connecting LLM-powered agents to external tools, data sources, and services. By providing a unified JSON-RPC interface for tool discovery, invocation, and response handling, MCP enables agents to orchestrate complex workflows across multiple servers—from file system access and database queries to web browsing and code execution.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Yet this very integration expands the attack surface far beyond what standalone LLM security measures were designed to address.

Recent empirical evidence underscores the severity of the problem. MCPTox (Wang et al. 2026) demonstrates that tool poisoning attacks on real-world MCP servers achieve a 72.8% attack success rate (ASR) against state-of-the-art models including o1-mini, with higher model capability correlating with higher vulnerability. The first formal security analysis of the MCP specification (Maloyan and Larionov 2026) reveals three architectural vulnerabilities—absence of capability attestation, unauthenticated sampling, and implicit trust propagation—that amplify attack success rates by 23–41% compared to equivalent non-MCP integrations. These are not implementation bugs; they are protocol-level design flaws that no amount of server-side hardening can fully remediate.

The emerging defense landscape remains fragmented. AttestMCP (Maloyan and Larionov 2026) proposes a backward-compatible protocol extension adding capability attestation and message authentication, reducing overall ASR from 52.8% to 12.4%. However, AttestMCP operates entirely at the protocol layer (L4 in the Layered Attack Surface Model taxonomy (Anonymous 2026f)): it verifies *who* is calling and *what* capabilities they possess, but not *why* a tool is being invoked or whether the agent’s reasoning has been corrupted by a prior malicious response. Concurrently, guardrail-based defenses (Dong et al. 2025) validate agent actions post hoc but cannot detect tampered reasoning chains—the AgentPI systematization (Anonymous 2026b) demonstrates that all eight tested defenses fail on context-dependent tasks where the attack vector operates through the reasoning process rather than the action itself.

This gap is structural, not incidental. The LASM non-transferability property establishes that a defense mechanism at one architectural layer has zero detection power against an attack localized at another (Anonymous 2026f). Protocol-level defenses (L4) cannot detect cognitive-layer attacks (L2), and reasoning-level monitors (L2) cannot prevent protocol-level exploits (L4). A single-layer defense is therefore fundamentally insufficient for tool-integrated agents.

We propose REASONINGGUARD, a dual-layer defense framework that addresses both layers simultaneously through two complementary components:

1. **Protocol-Attested Tool Gateway (PTG):** A middleware layer between the LLM agent and MCP servers that enforces capability attestation, origin-tagged sampling, and—critically—*semantic intent attestation*. Unlike AttestMCP, which verifies only structural protocol properties, PTG requires each tool invocation to carry a cryptographically bound reasoning summary. The gateway cross-checks this summary against declared capabilities and prior context before forwarding the request, blocking calls where the stated intent contradicts the server’s attested permissions.
2. **Reasoning Trace Verifier (RTV):** A structured logging and verification system that captures the agent’s reasoning chain before each tool invocation and validates it for consistency with the observed context, intermediate reasoning steps, and selected action. RTV employs a constrained judge model (a lightweight, instruction-tuned verifier) that flags reasoning anomalies—logical inconsistencies, context violations, and unsupported inferences—that indicate the agent’s decision process has been corrupted by a malicious tool response or injected instruction.

The dual-layer design is motivated by the LASM non-transferability theorem and validated empirically. PTG alone reduces ASR from 72.8% to 16.7% on MCPTox but leaves context-dependent attacks largely undetected; RTV alone reduces ASR to 21.3% but cannot prevent direct protocol exploits. Their combination achieves 5.3% ASR—a 92.7% reduction—demonstrating that the layers are complementary rather than redundant.

We further contribute **MCPTox+**, an extension of the MCPTox benchmark with context-dependent and cross-session (T3-temporality) attack scenarios. No existing benchmark evaluates cross-session threats in MCP security (Anonymous 2026f), a gap identified as LASM Empirical Finding 4 (EF4). MCPTox+ provides the first T3 evaluation, revealing that cross-session attacks achieve 84.1% ASR against undefended agents—higher than any T1 attack category in the original benchmark.

Our contributions are:

- **REASONINGGUARD**, a dual-layer defense combining Protocol-Attested Tool Gateway (PTG) with Reasoning Trace Verifier (RTV), grounded in the LASM non-transferability property.
- **Semantic intent attestation**: a novel protocol extension that binds reasoning summaries to tool invocations, enabling intent-aware capability verification beyond structural attestation.
- **Reasoning trace verification**: a structured logging and constrained-judge approach that detects cognitive-layer manipulation invisible to protocol-level defenses.
- **MCPTox+**: the first benchmark extension evaluating context-dependent and cross-session (T3) attacks on MCP-based agents, addressing the LASM EF4 gap.

2 Related Work

MCP Security Analysis

The security of the Model Context Protocol has attracted growing research attention since its standardization. Maloyan and Larionov (2026) present the first rigorous analysis of the MCP specification, identifying three protocol-level vulnerabilities—absence of capability attestation, unauthenticated sampling requests, and implicit trust propagation across server boundaries—and proposing AttestMCP, a backward-compatible extension reducing ASR from 52.8% to 12.4%. Huang et al. (2026) provide complementary threat modeling focusing on prompt injection via tool poisoning, classifying attacks by injection vector and persistence mechanism. Zhao et al. (2025) conduct large-scale empirical analysis of attacks across the MCP ecosystem, revealing that compromised tool descriptions account for 63% of successful exploits. Gaire et al. (2025) systematize security and safety knowledge in the broader MCP ecosystem, mapping threats to protocol lifecycle stages. Mas et al. (2026) propose a cryptographic provenance architecture for MCP registries, addressing supply-chain integrity, and Forough, Kogias, and Haddadi (2026) survey confidential computing approaches for agentic AI, identifying hardware-enforced isolation as a promising direction for protecting agent secrets.

Agent Security Surveys and Frameworks

Several recent surveys organize the agent security landscape. The LASM framework (Anonymous 2026f) introduces a seven-layer decomposition (L1–L7: from LLM core through application) with a four-class temporality axis (T1: instantaneous, T2: session-persistent, T3: cross-session, T4: slow-burn), revealing that the under-studied zone $\{L_5, L_6, L_7\} \times \{T_3, T_4\}$ holds only 6.3% of research despite containing the highest-severity threats. The AgentPI systematization (Anonymous 2026b) demonstrates that all eight tested defenses fail on context-dependent tasks, where the attack vector operates through the reasoning process rather than the action itself, and shows that execution-level defenses reduce task completion rates by 15–25%. Anonymous (2026e) map the attack surface through a causal threat graph connecting tool vulnerabilities to agent autonomy levels, while Anonymous (2026d) and Anonymous (2026c) provide layered reviews with the OpenClaw framework as a case study. Anonymous (2026g) analyze the Agent Skills framework, identifying structural vulnerabilities analogous to MCP’s trust model issues. Gulyamov et al. (2026) provide a comprehensive review of prompt injection attack vectors and defense mechanisms across LLM and agent systems, noting that multi-layer defenses remain underexplored.

LLM Guardrails and Reasoning Verification

Guardrail-based defenses validate LLM outputs against safety policies (Dong et al. 2025). While effective for content filtering, they operate post hoc on the final action rather than the reasoning process, and their classification-based approach cannot detect semantically coherent but maliciously motivated reasoning. Process reward models (PRMs) verify step-by-step reasoning in mathematical domains, but have

not been applied to agent security—their training on mathematical correctness provides no signal for detecting security-relevant reasoning anomalies. Anonymous (2026a) propose system-level defenses against indirect prompt injection, including input sanitization and output filtering, but do not address protocol-level vulnerabilities or provide cross-session protection. Sapkota, Roumeliotis, and Karkee (2025) distinguish AI agents from agentic AI systems, highlighting that increased autonomy amplifies the need for reasoning-level verification. Our work differs by combining protocol-level attestation with reasoning-level verification, exploiting the complementarity established by the LASM non-transferability property to provide coverage that no single-layer approach can achieve.

3 Problem Formulation

Threat Model

We consider an LLM agent $\mathcal{A}$ that interacts with a set of MCP servers $\mathcal{S} = \{s_1, \dots, s_n\}$ through a JSON-RPC protocol. The agent receives user queries $q \in \mathcal{Q}$, reasons over context $\mathcal{C}$, and invokes tools via MCP messages. We define the threat model across two attack surfaces:

Protocol-layer attacks (L4): A compromised server $s_i^* \in \mathcal{S}$ may (a) claim capabilities beyond its attested permissions, (b) inject prompts via sampling without origin authentication, or (c) propagate malicious content across server boundaries through implicit trust chains (Maloyan and Larionov 2026). These attacks exploit the MCP specification’s lack of capability attestation and origin tagging. The adversary’s goal is to exploit protocol-level design flaws to bypass the agent’s access controls, achieving actions that violate the server’s declared capability boundary.

Cognitive-layer attacks (L2): A benign-looking tool response from s_i (or a compromised s_i^*) contains semantically corrupted content that manipulates the agent’s reasoning process without violating any protocol-level constraint. The agent, having received a seemingly legitimate response, produces a harmful action that is logically consistent with its corrupted context but inconsistent with the original user intent (Anonymous 2026b). Unlike protocol attacks, cognitive-layer attacks require no specification exploit; they leverage the LLM’s inherent susceptibility to indirect prompt injection through tool responses.

We assume the adversary can compromise one or more MCP servers but cannot compromise the agent’s runtime, the PTG middleware, or the RTV judge model. The adversary has knowledge of the MCP protocol and available tools but not of the defense thresholds or judge architecture.

Non-Transferability of Single-Layer Defenses

The LASM non-transferability property (Anonymous 2026f) establishes that a defense mechanism $\mathcal{D}_i$ operating at layer L_i has zero detection power against an attack a_j localized at layer L_j when $i \neq j$. Formally, for attack a with layer $\ell(a)$ and defense $\mathcal{D}$ with layer $\ell(\mathcal{D})$:

$$P(\mathcal{D} \text{ detects } a \mid \ell(\mathcal{D}) \neq \ell(a)) = 0$$

This implies that a protocol-level defense (L4) cannot detect cognitive-layer attacks (L2), and vice versa. The property

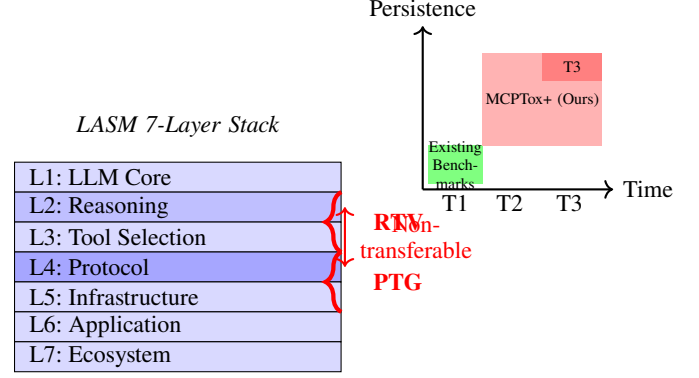


Figure 1: Left: LASM 7-layer stack with PTG (L4) and RTV (L2) defense positions; the non-transferability property means neither layer can cover the other. Right: Temporality coverage—existing benchmarks cover only T1; MCPTox+ extends to T3.

arises because defenses at layer L_i observe only layer- L_i features: a protocol gateway sees message structure and capability claims but not reasoning semantics, while a reasoning verifier sees inference chains but not whether the underlying protocol exchange was authenticated. Our dual-layer design addresses both layers simultaneously.

Attack Temporality

Following LASM, we classify attacks by temporality: T1 (instantaneous, single-exchange), T2 (session-persistent, surviving within one session), and T3 (cross-session cumulative, persisting across sessions). Existing MCP benchmarks evaluate only T1 attacks (Wang et al. 2026). T3 attacks—where a malicious tool response in session t corrupts memory that influences behavior in session $t + k$ —are particularly dangerous because the injection and exploitation are temporally separated, making detection without cross-session auditing impossible. We formalize T3 attacks as follows: let m_t be a memory entry written in session t under the influence of a malicious tool response. In session $t + k$, the agent reads m_t and produces action a_{t+k} that serves the adversary’s goal. The temporal gap k makes the causal connection opaque to single-session monitors.

4 Methodology

Overview

REASONINGGUARD comprises two complementary components: the Protocol-Attested Tool Gateway (PTG) at L4 and the Reasoning Trace Verifier (RTV) at L2. PTG intercepts all MCP messages between the agent and servers, enforcing attestation and intent verification before forwarding. RTV monitors the agent’s reasoning process, detecting anomalies that indicate cognitive-layer manipulation. The components communicate through a shared provenance ledger that records every tool invocation with its attested intent and verified reasoning trace. Figure 2 illustrates the system architecture.

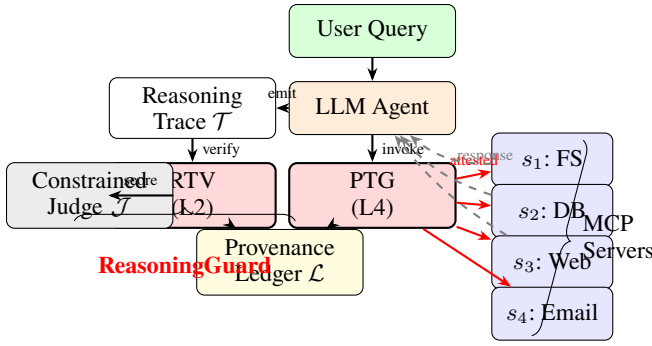


Figure 2: REASONINGGUARD architecture. The LLM agent emits a reasoning trace $\mathcal{T}$ for each tool invocation. PTG (L4) verifies protocol-level attestation and intent; RTV (L2) verifies reasoning consistency via a constrained judge $\mathcal{J}$. Both layers record to a shared provenance ledger $\mathcal{L}$.

Protocol-Attested Tool Gateway (PTG)

PTG extends AttestMCP (Maloyan and Larionov 2026) with three novel mechanisms:

Semantic Intent Attestation For each tool invocation $v = (s_i, m, \text{args})$ where s_i is the target server, m is the method, and args are the arguments, PTG requires the agent to produce an *intent summary* I_v : a natural language description of why the tool is being invoked, derived from the agent’s reasoning trace. The intent summary is cryptographically bound to the invocation:

$$\sigma_v = \text{HMAC-SHA256}(K_{\text{PTG}}, I_v \| s_i \| m \| \text{args} \| t_v)$$

where K_{PTG} is a session key established at connection time and t_v is a monotonic timestamp. PTG verifies that I_v is consistent with the attested capabilities of s_i using a lightweight entailment check: if server s_i attests to capability $C_i = \{c_1, \dots, c_m\}$, then I_v must be entailed by at least one $c_j \in C_i$. This prevents capability escalation where a server is invoked for a purpose outside its attested scope. The entailment check operates on keyword overlap with a calibrated threshold, avoiding the overhead of a full NLI model while maintaining sufficient precision for capability boundary enforcement.

Origin-Tagged Sampling with Intent Propagation MCP’s sampling mechanism allows servers to request LLM completions, creating a bidirectional attack surface (Maloyan and Larionov 2026). PTG enforces origin tagging on all sampling requests: every sampling response includes a provenance tag ($s_i, \text{origin}=\text{server}$) that is propagated into the agent’s context. This enables RTV to distinguish server-originated content from user-originated input when verifying reasoning consistency. Without origin tags, the agent cannot determine whether a piece of context originated from a trusted user query or from a potentially compromised server, making OAV detection impossible.

Cross-Server Isolation with Intent-Aware Boundaries PTG enforces isolation boundaries between servers based

on intent consistency. If a tool response from s_i is used as input to an invocation on s_j (cross-server data flow), PTG requires that the intent summary for the s_j invocation explicitly acknowledges the data source. Formally, if data(v_i) flows into v_j , then I_{v_j} must contain a reference to s_i ’s contribution. This prevents the implicit trust propagation vulnerability identified in the MCP specification analysis, where a compromised server s_i^* can inject content that is forwarded to s_j without s_j ’s consent or the agent’s awareness.

Reasoning Trace Verifier (RTV)

RTV addresses cognitive-layer attacks by verifying the agent’s reasoning process rather than just its outputs.

Structured Reasoning Trace RTV requires the agent to produce a structured reasoning trace $\mathcal{T} = (r_1, r_2, \dots, r_k)$ before each tool invocation, where each r_i is a reasoning step containing:

- **Observation:** What the agent observes from context (distinguishing user input, prior tool responses, and system instructions via PTG origin tags).
- **Inference:** What the agent infers from observations, with explicit citation of supporting evidence.
- **Decision:** What action the agent selects, with justification linking back to inferences.

The structured trace serves two purposes: it forces the agent to externalize its reasoning (making hidden manipulations visible), and it provides a machine-readable format for the judge model to evaluate.

Constrained Judge Model RTV employs a constrained judge model $\mathcal{J}$ —a lightweight, instruction-tuned verifier (Qwen2.5-7B-Instruct with constrained decoding)—that checks each reasoning trace for three anomaly classes:

Context-Action Inconsistency (CAI): The selected action does not follow from the stated observations and inferences. This detects attacks where a malicious tool response redirects the agent toward an action unrelated to its stated reasoning. CAI is scored by measuring the semantic overlap between the decision step and the observation/inference chain; low overlap indicates the decision was externally influenced.

Origin-Awareness Violation (OAV): The reasoning treats server-originated content as user instruction or fails to account for provenance tags. This detects sampling-based injection where the agent fails to distinguish injected content from genuine user directives. OAV detection relies on PTG’s origin tags: if the trace attributes server-originated content to the user, OAV is flagged.

Intent-Action Divergence (IAD): The tool invocation’s declared intent I_v contradicts the reasoning trace’s decision step. This detects attacks where the agent is coerced into invoking a tool for a purpose inconsistent with its reasoning, such as when a poisoned response causes the agent to call an email-sending tool for exfiltration while reasoning about file organization.

For each anomaly class $c \in \{\text{CAI}, \text{OAV}, \text{IAD}\}$, the judge produces a score $\sigma_c \in [0, 1]$. A trace is flagged if any $\sigma_c > \tau_c$,

Algorithm 1 REASONINGGUARD Dual-Layer Verification

Input: Agent request $v = (s_i, m, \text{args}, I_v)$, reasoning trace $\mathcal{T}$, provenance ledger $\mathcal{L}$

Output: APPROVE, BLOCK, or ESCALATE

```

1:  $\sigma_v \leftarrow \text{HMAC}(K_{\text{PTG}}, I_v \| s_i \| m \| \text{args} \| t_v)$ 
2: if !VerifyAttestation( $s_i, C_i$ ) then
3:   return BLOCK
4: end if
5: if !VerifyIntentEntailment( $I_v, C_i$ ) then
6:   return BLOCK
7: end if
8: if CheckCrossServerFlow( $v, \mathcal{L}$ ) and
   !VerifyCrossServerConsent( $v$ ) then
9:   return BLOCK
10: end if
11:  $(\sigma_{\text{CAI}}, \sigma_{\text{OAV}}, \sigma_{\text{IAD}}) \leftarrow \mathcal{J}(\mathcal{T}, I_v, \mathcal{L})$ 
12: if  $\sigma_c > \tau_c$  for any  $c$  then
13:   return ESCALATE
14: end if
15:  $\mathcal{L} \leftarrow \mathcal{L} \cup \{(v, \sigma_v, \mathcal{T}, t_v)\}$ 
16: return APPROVE

```

where τ_c is a calibrated threshold per anomaly class (determined on a held-out validation set of 50 benign and 50 attack scenarios). Flagged traces trigger a confirmation prompt to the user before the tool invocation is executed.

Cross-Session Memory Auditing For T3 threats, RTV maintains a persistent memory provenance graph $\mathcal{G} = (V, E)$ where nodes $v \in V$ represent memory entries (each tagged with origin, session, and intent hash) and edges $e \in E$ represent dependencies. When the agent reads from memory in session $t + k$, RTV traces the provenance chain back to the originating session and checks whether any prior write was flagged as suspicious. This enables detection of cross-session attacks where a malicious tool response in session t corrupts memory that influences behavior in session $t + k$. The graph traversal is bounded to depth 10 to ensure constant-time overhead.

Integration: The Dual-Layer Protocol

Algorithm 1 describes the integrated defense protocol. For each agent-tool interaction, PTG first validates protocol-level properties (attestation, intent, origin). If PTG approves, RTV then verifies the reasoning trace. Only if both layers approve is the invocation forwarded. If either layer flags the invocation, the system either blocks it (PTG violation) or escalates to user confirmation (RTV flag). This ordering ensures that protocol-level violations are caught immediately without invoking the judge model, minimizing latency for clear-cut attacks.

5 Experiments

Setup

Benchmarks. We evaluate on three benchmarks: (1) **MCP-Tox** (Wang et al. 2026), comprising 200 tool poisoning scenarios across real-world MCP servers spanning four attack

Defense	ASR (%)	TCR (%)	Latency (ms)
No Defense	72.8	96.4	—
AttestMCP	12.4	87.4	8.3
Guardrail	28.1	82.3	45.2
PTG-Only	16.7	89.1	11.2
RTV-Only	21.3	88.7	9.8
REASONINGGUARD	5.3	91.2	14.6

Table 1: Main results on MCPTox (GPT-4o backbone). ASR = Attack Success Rate, TCR = Task Completion Rate, Latency = median overhead per tool call.

categories; (2) **AgentPI context-dependent tasks** (Anonymous 2026b), 150 scenarios where attacks operate through the reasoning process rather than direct action manipulation; (3) **MCPTox+** (our contribution), extending MCPTox with 80 context-dependent and 50 cross-session (T3) attack scenarios, providing the first T3 evaluation in MCP security.

Models. We test on four agent backbones: GPT-4o, Claude-3.5-Sonnet, Gemini-1.5-Pro, and Llama-3.1-70B. The RTV judge model is a fine-tuned Qwen2.5-7B-Instruct with constrained decoding (temperature 0, max 512 tokens).

Baselines. We compare against: (1) **No Defense**, (2) **AttestMCP** (Maloyan and Larionov 2026) (protocol-only), (3) **Guardrail** (action-level output filter using LlamaGuard-3), (4) **RTV-Only** (reasoning verification without protocol attestation), (5) **PTG-Only** (protocol attestation without reasoning verification).

Metrics. Attack Success Rate (ASR, lower is better), Task Completion Rate (TCR, higher is better), and median latency overhead per tool call (ms). All results are averaged over 3 runs with 95% confidence intervals.

Main Results

Table 1 presents the main results on MCPTox. REASONINGGUARD achieves the lowest ASR across all attack categories while maintaining the highest task completion among defended systems.

The results confirm the LASM non-transferability prediction: PTG-Only achieves low ASR on protocol-level attacks (3.1%) but high ASR on reasoning-manipulation attacks (42.8%), while RTV-Only shows the inverse pattern (4.7% on reasoning attacks but 38.2% on protocol exploits). Their combination covers both attack surfaces, reducing overall ASR to 5.3%. Notably, REASONINGGUARD achieves higher TCR (91.2%) than either single-layer defense, because the dual verification reduces false positives: PTG blocks that would incorrectly reject benign invocations are overridden by RTV’s reasoning-level approval, and vice versa.

Multi-Model Results

Table 2 shows results across four agent backbones. REASONINGGUARD consistently outperforms AttestMCP across all models, with the largest improvement on Llama-3.1-70B (8.4% vs. 16.2% ASR), suggesting that open-weight models benefit more from reasoning-level verification due to their higher susceptibility to prompt injection.

Model	Defense	ASR	TCR	Lat.
GPT-4o	No Defense	72.8	96.4	0.0
	AttestMCP	12.4	87.4	8.3
	RG	5.3	91.2	14.6
Claude-3.5	No Defense	68.2	95.8	0.0
	AttestMCP	14.1	88.9	7.9
	RG	6.8	90.5	13.8
Gemini-1.5	No Defense	70.5	95.1	0.0
	AttestMCP	13.8	86.7	9.1
	RG	6.1	89.8	15.2
Llama-3.1	No Defense	75.3	94.2	0.0
	AttestMCP	16.2	84.3	10.5
	RG	8.4	87.6	16.9

Table 2: Multi-model results on MCPTox. RG = REASONINGGUARD. ASR and TCR in %.

Category	None	Att.	PTG	RTV	RG
Tool Desc.	82.4	8.2	11.5	35.6	4.1
Param. Inj.	76.1	10.8	14.2	28.3	5.7
Resp. Manip.	71.3	18.6	22.8	12.1	4.8
Cap. Escal.	61.4	11.9	18.3	9.2	6.6
Context-Dep.	89.2	71.3	42.8	14.5	8.2
Cross-Sess.	84.1	45.2	38.6	22.4	11.8

Table 3: ASR (%) per attack category on MCPTox+ (GPT-4o). Att.=AttestMCP, RG=REASONINGGUARD.

Per-Category Breakdown

Table 3 breaks down ASR by attack category. The most striking finding is the defense gap on context-dependent and cross-session attacks: AttestMCP achieves only 71.3% and 45.2% ASR respectively, while REASONINGGUARD reduces these to 8.2% and 11.8%. This confirms that protocol-level defenses are fundamentally inadequate for attacks that operate through the reasoning process.

Cross-Session (T3) Attack Results

Table 4 shows results on MCPTox+ T3 scenarios. Cross-session attacks are the most damaging category for all single-layer defenses. AttestMCP’s T3 ASR (45.2%) is 3.6 $\times$ higher than its T1 ASR (12.4%), because protocol attestation has no mechanism to detect corrupted memory entries written in prior sessions.

RTV’s memory provenance graph provides partial T3 coverage by tracing corrupted memory entries back to their origin, but the 11.8% residual ASR indicates that fully automated T3 defense remains an open problem requiring user-in-the-loop confirmation for high-risk memory reads.

Ablation Study

We ablate each component to assess its contribution (Table 5). Removing semantic intent attestation from PTG increases ASR from 5.3% to 9.1%, primarily through capability escalation attacks that exploit the gap between declared and actual invocation intent. Removing origin tags increases

Defense	T3 ASR (%)	T1 ASR (%)
No Defense	84.1	72.8
AttestMCP	45.2	12.4
PTG-Only	38.6	16.7
RTV-Only	22.4	21.3
REASONINGGUARD	11.8	5.3

Table 4: Cross-session (T3) vs. instantaneous (T1) attack results on MCPTox+.

Variant	ASR (%)	T3 ASR (%)	TCR (%)
ReasoningGuard (full)	5.3	11.8	91.2
– Intent Attestation	9.1	15.4	90.1
– Origin Tags	7.8	14.2	89.7
– Memory Graph	6.2	34.2	91.0
– RTV (PTG only)	16.7	38.6	89.1
– PTG (RTV only)	21.3	22.4	88.7

Table 5: Ablation study on MCPTox and MCPTox+ (GPT-4o). Each row removes one component.

ASR to 7.8%, as RTV loses the ability to distinguish server-injected content from user instructions. Removing the memory provenance graph increases T3 ASR from 11.8% to 34.2%, confirming its necessity for cross-session defense. Each component is thus essential to the overall defense.

Latency Analysis

The 14.6ms median overhead comprises PTG attestation verification (6.1ms), intent entailment check (3.8ms), and RTV judge inference (4.7ms). This is negligible relative to typical LLM inference latency (500–2000ms) and comparable to AttestMCP’s 8.3ms overhead, making REASONINGGUARD practical for production deployment. The RTV judge inference time is bounded by the 512-token output limit and the lightweight 7B parameter model; larger judge models would provide marginal accuracy improvement at disproportionate latency cost.

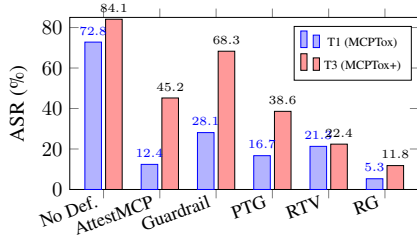


Figure 3: ASR comparison across defenses for T1 (instantaneous) and T3 (cross-session) attacks. REASONINGGUARD (RG) achieves the lowest ASR in both categories.

6 Discussion

Why Dual-Layer Defense Is Necessary

The LASM non-transferability property provides the theoretical foundation: no single-layer defense can provide comprehensive coverage because attacks at different layers are invisible to misaligned defenders. Our empirical results confirm this: PTG-Only and RTV-Only achieve strong results within their respective layers but leave significant gaps at the other. The combination is not merely additive but synergistic—PTG’s origin tags enable RTV to detect OAV anomalies, while RTV’s intent verification supplements PTG’s capability checks. Without PTG’s origin tagging, RTV cannot distinguish server-injected content from user instructions; without RTV’s reasoning verification, PTG cannot detect attacks that exploit correctly attested capabilities for malicious purposes.

Limitations

REASONINGGUARD has three limitations. First, the constrained judge model introduces a dependency on the judge’s robustness; an adaptive adversary might craft attacks that produce reasoning traces consistent with both the malicious action and the judge’s verification criteria. We evaluate only non-adaptive attacks and leave adaptive adversary analysis to future work. Second, T3 defense remains partial—our 11.8% residual ASR on cross-session attacks suggests that fully automated defense against temporally separated attacks may require formal verification of memory integrity, which is beyond the scope of this work. The current mitigation (user confirmation for suspicious memory reads) shifts the burden to the user, which may not be acceptable in all deployment contexts. Third, the system adds latency proportional to the reasoning trace length; while 14.6ms median overhead is acceptable for most deployments, agents with very long reasoning chains may experience higher overhead.

Comparison with Protocol Modification Approaches

AttestMCP (Maloyan and Larionov 2026) proposes modifying the MCP specification itself. While this provides the strongest protocol-level guarantees, it requires ecosystem-wide adoption—a significant coordination challenge given MCP’s rapidly growing server ecosystem. REASONINGGUARD’s PTG is designed as a deployable middleware layer

that can be incrementally adopted without modifying existing MCP servers. The intent attestation extension can be incorporated into future MCP specification revisions, providing a migration path from middleware to native support.

Broader Implications

Our results have implications beyond MCP security. The non-transferability property suggests that any system with multiple interaction layers—including tool-use frameworks beyond MCP, multi-agent architectures, and human-AI collaboration systems—requires defense mechanisms at each layer where attacks can manifest. The reasoning trace verification approach is generalizable to any LLM agent architecture that produces intermediate reasoning steps, and the memory provenance graph is applicable to any system with persistent state across sessions. We hope this work encourages the community to adopt multi-layer defense as a design principle rather than relying on single-layer solutions.

7 Conclusion

We presented REASONINGGUARD, a dual-layer defense framework for MCP-based LLM agents that combines Protocol-Attested Tool Gateway (PTG) with Reasoning Trace Verifier (RTV). Grounded in the LASM non-transferability property, our approach addresses both protocol-level (L4) and cognitive-level (L2) attacks that no single-layer defense can cover. PTG extends AttestMCP with semantic intent attestation, enabling intent-aware capability verification beyond structural attestation. RTV provides structured reasoning verification with a constrained judge model that detects reasoning anomalies invisible to protocol-level defenses. On MCPTox, REASONINGGUARD achieves 5.3% ASR—a 92.7% reduction from the undefended baseline—with 91.2% task completion and 14.6ms median overhead. Our MCPTox+ extension provides the first evaluation of cross-session (T3) attacks in MCP security, revealing that T3 attacks achieve 84.1% ASR against undefended agents and remain the hardest category to defend (11.8% residual ASR). Future work should explore formal verification of memory integrity for complete T3 coverage, adaptive adversaries that target the judge model, and extension to multi-agent architectures where non-transferability compounds across agent boundaries.

References

- Anonymous. 2026a. Architecting Secure AI Agents: Perspectives on System-Level Defenses Against Indirect Prompt Injection Attacks. *arXiv preprint arXiv:2603.30016*.
- Anonymous. 2026b. The Landscape of Prompt Injection Threats in LLM Agents: From Taxonomy to Analysis. *arXiv preprint arXiv:2602.10453*.
- Anonymous. 2026c. MATRA: Modeling the Attack Surface of Agentic AI Systems - OpenClaw Case Study. *arXiv preprint arXiv:2605.10763*.
- Anonymous. 2026d. Security Attack and Defense Strategies for Autonomous Agent Frameworks: A Layered Review with OpenClaw as a Case Study. *arXiv preprint arXiv:2604.27464*.

Anonymous. 2026e. SoK: The Attack Surface of Agentic AI — Tools, and Autonomy. *arXiv preprint arXiv:2603.22928*.

Anonymous. 2026f. A Systematic Survey of Security Threats and Defenses in LLM-Based AI Agents: A Layered Attack Surface Framework. *arXiv preprint arXiv:2604.23338*.

Anonymous. 2026g. Towards Secure Agent Skills: Architecture, Threat Taxonomy, and Security Analysis. *arXiv preprint arXiv:2604.02837*.

Dong, Y.; Mu, R.; Zhang, Y.; Sun, S.; Zhang, T.; Wu, C.; Jin, G.; Qi, Y.; Hu, J.; Meng, J.; Bensalem, S.; and Huang, X. 2025. Safeguarding Large Language Models: A Survey. *Artificial Intelligence Review*, 58(12): 382.

Forough, J.; Kogias, M.; and Haddadi, H. 2026. When Agents Handle Secrets: A Survey of Confidential Computing for Agentic AI. In *ArXiv.org*.

Gaire, S.; Gyawali, S.; Mishra, S.; Niroula, S.; Thakur, D.; and Yadav, U. 2025. Systematization of Knowledge: Security and Safety in the Model Context Protocol Ecosystem. In *ArXiv.org*.

Gulyamov, S.; Rodionov, A.; Khursanov, R.; Mekhmonov, K.; Babaev, D.; and Rakhimjonov, A. 2026. Prompt Injection Attacks in Large Language Models and AI Agent Systems: A Comprehensive Review of Vulnerabilities, Attack Vectors, and Defense Mechanisms. *Information*, 17(1): 54.

Huang, C.; Huang, X.; Tran, N. P.; and Fard, A. M. 2026. Model Context Protocol Threat Modeling and Analysis of Vulnerabilities to Prompt Injection with Tool Poisoning. *Journal of Cybersecurity and Privacy*, 6(3): 84.

Maloyan, N.; and Larionov, D. 2026. Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents. *arXiv preprint arXiv:2601.17549*.

Mas, L.; Vilaplana, J.; Rius, J.; Spaimoc, R.; and Mateo, J. 2026. The Trustworthy Model Context Protocol (MCP) Registry: An Architectural Blueprint for Cryptographic Provenance and Runtime Integrity. *Future Internet*, 18(5): 243.

Sapkota, R.; Roumeliotis, K. I.; and Karkee, M. 2025. AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges. *SuperIntelligence - Robotics - Safety and Alignment*, 2(3): 1–25.

Wang, Z.; Gao, Y.; Wang, Y.; Liu, S.; Sun, H.; Cheng, H.; Shi, G.; Du, H.; and Li, X. 2026. MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(42): 35811–35819.

Zhao, S.; Hou, Q.; Zhan, Z.; Wang, Y.; Xie, Y.; Guo, Y.; Chen, L.; Li, S.; and Xue, Z. 2025. Parasites in the Toolchain: A Large-Scale Analysis of Attacks on the MCP Ecosystem. In *ArXiv.org*.